

e_coli_flux_balance_analysis

July 3, 2026

Our project will simulate how fast *E. coli* grows under normal conditions, and then we will “play God” by altering its environment (suffocating it) and mutating its genetics (deleting a gene) to see how the mathematics predict its survival.

To do this, we will use **Python** and a famous scientific library called **COBRApy** (COntstraint-Based Reconstruction and Analysis).

0.0.1 Phase 1: Setting up the Virtual Laboratory

If you are running this yourself, first install the library:

```
[1]: pip install cobra
```

```
Collecting cobra
  Downloading cobra-0.31.1-py2.py3-none-any.whl.metadata (9.3 kB)
Collecting appdirs~=1.4 (from cobra)
  Using cached appdirs-1.4.4-py2.py3-none-any.whl.metadata (9.0 kB)
Collecting depinfo~=2.2 (from cobra)
  Downloading depinfo-2.2.0-py3-none-any.whl.metadata (3.8 kB)
Collecting future (from cobra)
  Downloading future-1.0.0-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: httpx~=0.24 in ./venv/lib/python3.12/site-packages (from cobra) (0.28.1)
Collecting numpy>=1.13 (from cobra)
  Downloading
numpy-2.5.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
(6.6 kB)
Collecting optlang~=1.9 (from cobra)
  Downloading optlang-1.9.1-py2.py3-none-any.whl.metadata (8.2 kB)
Collecting pandas<3.0,>=1.0 (from cobra)
  Using cached pandas-2.3.3-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (91 kB)
Collecting pydantic>=1.6 (from cobra)
  Using cached pydantic-2.13.4-py3-none-any.whl.metadata (109 kB)
Collecting python-libsbml~=5.19 (from cobra)
  Downloading python_libsbml-5.21.1-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (666 bytes)
Collecting rich>=8.0 (from cobra)
  Using cached rich-15.0.0-py3-none-any.whl.metadata (18 kB)
```

```

Collecting ruamel.yaml~=0.16 (from cobra)
  Downloading ruamel_yaml-0.19.1-py3-none-any.whl.metadata (16 kB)
Collecting swiglpk (from cobra)
  Downloading swiglpk-5.0.13-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(5.7 kB)
Requirement already satisfied: anyio in ./venv/lib/python3.12/site-packages
(from httpx~=0.24->cobra) (4.14.1)
Requirement already satisfied: certifi in ./venv/lib/python3.12/site-packages
(from httpx~=0.24->cobra) (2026.6.17)
Requirement already satisfied: httpcore==1.* in ./venv/lib/python3.12/site-
packages (from httpx~=0.24->cobra) (1.0.9)
Requirement already satisfied: idna in ./venv/lib/python3.12/site-packages
(from httpx~=0.24->cobra) (3.18)
Requirement already satisfied: h11>=0.16 in ./venv/lib/python3.12/site-packages
(from httpcore==1.*->httpx~=0.24->cobra) (0.16.0)
Collecting sympy>=1.12.0 (from optlang~=1.9->cobra)
  Using cached sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in
./venv/lib/python3.12/site-packages (from pandas<3.0,>=1.0->cobra)
(2.9.0.post0)
Collecting pytz>=2020.1 (from pandas<3.0,>=1.0->cobra)
  Downloading pytz-2026.2-py2.py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: tzdata>=2022.7 in ./venv/lib/python3.12/site-
packages (from pandas<3.0,>=1.0->cobra) (2026.2)
Collecting annotated-types>=0.6.0 (from pydantic>=1.6->cobra)
  Using cached annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.46.4 (from pydantic>=1.6->cobra)
  Using cached pydantic_core-2.46.4-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.6 kB)
Requirement already satisfied: typing-extensions>=4.14.1 in
./venv/lib/python3.12/site-packages (from pydantic>=1.6->cobra) (4.16.0)
Collecting typing-inspection>=0.4.2 (from pydantic>=1.6->cobra)
  Using cached typing_inspection-0.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting markdown-it-py>=2.2.0 (from rich>=8.0->cobra)
  Using cached markdown_it_py-4.2.0-py3-none-any.whl.metadata (7.4 kB)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
./venv/lib/python3.12/site-packages (from rich>=8.0->cobra) (2.20.0)
Collecting mdurl~=0.1 (from markdown-it-py>=2.2.0->rich>=8.0->cobra)
  Using cached mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages
(from python-dateutil>=2.8.2->pandas<3.0,>=1.0->cobra) (1.17.0)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.12.0->optlang~=1.9->cobra)
  Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Downloading cobra-0.31.1-py2.py3-none-any.whl (1.2 MB)

```

```

1.2/1.2 MB 773.7 kB/s eta 0:00:00 1m753.6 kB/s eta
0:00:01

```

```

Using cached appdirs-1.4.4-py2.py3-none-any.whl (9.6 kB)
Downloading depinfo-2.2.0-py3-none-any.whl (24 kB)
Downloading
numpy-2.5.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7
MB)

16.7/16.7 MB 996.1 kB/s eta 0:00:00eta
0:00:01[36m0:00:010m
Downloading optlang-1.9.1-py2.py3-none-any.whl (141 kB)

142.0/142.0 kB 3.7 MB/s eta 0:00:00[31m7.5 MB/s
eta 0:00:01
Using cached
pandas-2.3.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.4
MB)
Using cached pydantic-2.13.4-py3-none-any.whl (472 kB)
Using cached
pydantic_core-2.46.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
(2.1 MB)
Downloading python_libsbml-5.21.1-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (8.0 MB)

8.0/8.0 MB 1.0 MB/s eta 0:00:00m eta
0:00:01[36m0:00:01
Using cached rich-15.0.0-py3-none-any.whl (310 kB)
Downloading ruamel_yaml-0.19.1-py3-none-any.whl (118 kB)

118.1/118.1 kB 1.8 MB/s eta 0:00:00MB/s eta
0:00:01
Downloading swiglpk-5.0.13-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (2.4 MB)

2.4/2.4 MB 778.5 kB/s eta 0:00:001m760.5 kB/s eta
0:00:01
Downloading future-1.0.0-py3-none-any.whl (491 kB)

491.3/491.3 kB 699.3 kB/s eta 0:00:001m811.6 kB/s
eta 0:00:01
Using cached annotated_types-0.7.0-py3-none-any.whl (13 kB)
Using cached markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Downloading pytz-2026.2-py2.py3-none-any.whl (510 kB)

510.1/510.1 kB 438.9 kB/s eta 0:00:001m431.0 kB/s
eta 0:00:01
Using cached sympy-1.14.0-py3-none-any.whl (6.3 MB)
Using cached typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Using cached mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Using cached mpmath-1.3.0-py3-none-any.whl (536 kB)

```

Installing collected packages: swiglpk, pytz, python-libsbml, mpmath, appdirs, typing-inspection, sympy, ruamel.yaml, pydantic-core, numpy, mdurl, future, depinfo, annotated-types, pydantic, pandas, optlang, markdown-it-py, rich, cobra
Successfully installed annotated-types-0.7.0 appdirs-1.4.4 cobra-0.31.1
depinfo-2.2.0 future-1.0.0 markdown-it-py-4.2.0 mdurl-0.1.2 mpmath-1.3.0
numpy-2.5.0 optlang-1.9.1 pandas-2.3.3 pydantic-2.13.4 pydantic-core-2.46.4
python-libsbml-5.21.1 pytz-2026.2 rich-15.0.0 ruamel.yaml-0.19.1 swiglpk-5.0.13
sympy-1.14.0 typing-inspection-0.4.2

Note: you may need to restart the kernel to use updated packages.

Now, let's load our virtual organism. We will use the “**E. coli Core Model**”. This is a simplified blueprint of *E. coli*'s central metabolism containing 95 reactions and 72 metabolites—perfect for learning.

```
[2]: import cobra

# 1. Load the core E. coli blueprint
model = cobra.io.load_model("textbook")

print(f"Number of Metabolites: {len(model.metabolites)}")
print(f"Number of Reactions: {len(model.reactions)}")
print(f"Objective (Goal): {model.objective.expression}")
```

Number of Metabolites: 72

Number of Reactions: 95

Objective (Goal): 1.0*Biomass_Ecoli_core - 1.0*Biomass_Ecoli_core_reverse_2cdba

Expected Output: > Number of Metabolites: 72 > Number of Reactions: 95 > Objective (Goal):
1.0 * Biomass_Ecoli_core - 1.0 * Biomass_Ecoli_core_reverse

0.0.2 Phase 2: Experiment 1 - The “All-You-Can-Eat” Buffet (Aerobic Growth)

By default, the core model assumes the bacterium is in a warm petri dish with plenty of **glucose** and **oxygen** (Aerobic conditions). Let's run Flux Balance Analysis (FBA) to find the maximum possible growth rate.

```
[3]: # Run Flux Balance Analysis (FBA)
solution = model.optimize()

# Print the predicted growth rate (Biomass flux)
print(f"Aerobic Growth Rate: {solution.objective_value:.3f} per hour")

# Let's check how much Glucose and Oxygen the cell is consuming
print(f"Glucose consumed: {solution.fluxes['EX_glc__D_e']:.2f}")
print(f"Oxygen consumed: {solution.fluxes['EX_o2_e']:.2f}")
```

Aerobic Growth Rate: 0.874 per hour

Glucose consumed: -10.00

Oxygen consumed: -21.80

Expected Output: > Aerobic Growth Rate: 0.874 per hour > Glucose consumed: -10.00 > Oxygen consumed: -21.80

(Note: Consumption fluxes are negative because they are leaving the environment and entering the cell). **What this means:** Under perfect conditions, our *E. coli* mathematically splits and doubles roughly every 45-50 minutes, consuming 10 units of glucose and 21.8 units of oxygen.

0.0.3 Phase 3: Experiment 2 - Suffocating the Cell (Anaerobic Growth)

What happens if we seal the petri dish so no oxygen gets in? We need to change the **constraints** of our mathematical model. We will set the maximum oxygen uptake limit to zero.

```
[4]: # Change the lower bound of Oxygen exchange to 0 (No oxygen enters)
model.reactions.EX_o2_e.lower_bound = 0

# Re-run FBA with the new constraints
anaerobic_solution = model.optimize()

print(f"Anaerobic Growth Rate: {anaerobic_solution.objective_value:.3f} per_
↪hour")

# Let's see if the cell is producing a new byproduct to survive
print(f"Ethanol produced: {anaerobic_solution.fluxes['EX_etoh_e']:.2f}")
print(f"Acetate produced: {anaerobic_solution.fluxes['EX_ac_e']:.2f}")
```

Anaerobic Growth Rate: 0.212 per hour
Ethanol produced: 8.28
Acetate produced: 8.50

Expected Output: > Anaerobic Growth Rate: 0.212 per hour > Ethanol produced: 8.28 > Acetate produced: 8.58

What this means: This is the magic of FBA! Because oxygen is gone, the computer realizes the cell can no longer use cellular respiration (which makes a ton of ATP). To satisfy the math ($S \times v = 0$), the cell *must* route the flux through fermentation instead. The growth rate drops dramatically (from 0.874 to 0.212), and the computer correctly predicts that the cell will start secreting **ethanol and acetate** (acids/alcohol) as waste!

0.0.4 Phase 4: Experiment 3 - The Genetic Knockout & The Metabolic Whirlpool

Now, let's play genetic engineer. We will delete a vital glycolysis gene that produces the enzyme *Phosphofructokinase (PFK)*.

To simulate this accurately, we will use an advanced tool called **Parsimonious FBA (pFBA)**. Normal FBA only maximizes growth rate, which can sometimes result in impossible math glitches. pFBA does a two-step optimization: First, it maximizes the growth rate. Second, it finds the route that requires the **absolute minimum chemical effort** (minimizing the sum of all internal fluxes) to achieve that growth. Cells are naturally efficient, so pFBA acts closer to real biology.

```
[11]: # Import the specialized pFBA tool
from cobra.flux_analysis import pfba

# 1. Reset the model to fresh (ACTUALLY turn the oxygen back on!)
model.reactions.EX_o2_e.lower_bound = -1000.0

with model:
    # 2. Find the PFK reaction and force its flux to 0 (simulate a knockout)
    model.reactions.PFK.knock_out()

    # 3. Run Parsimonious FBA (pFBA) on the mutant
    parsimonious_solution = pfba(model)

    # In pFBA, the mathematical objective becomes the "Sum of all fluxes"
    # To get the actual growth rate, we check the Biomass flux directly:
    print(f"Mutant Growth Rate: {parsimonious_solution.
↪fluxes['Biomass_Ecoli_core']:.3f} per hour")
    print(f"Total Chemical Effort (Sum of Fluxes): {parsimonious_solution.
↪objective_value:.3f}")

    # Let's check how the cell is bypassing the broken enzyme
    print(f"Flux through normal pathway (PFK): {parsimonious_solution.
↪fluxes['PFK']:.2f}")
    print(f"Flux through alternate pathway (G6PDH2r): {parsimonious_solution.
↪fluxes['G6PDH2r']:.2f}")
```

```
Mutant Growth Rate: 0.704 per hour
Total Chemical Effort (Sum of Fluxes): 680.110
Flux through normal pathway (PFK): 0.00
Flux through alternate pathway (G6PDH2r): 27.90
```

Expected Output: > Mutant Growth Rate: 0.704 per hour > Total Chemical Effort (Sum of Fluxes): 680.110 > Flux through normal pathway (PFK): 0.00 > Flux through alternate pathway (G6PDH2r): 27.90

The Biological Explanation: Stoichiometric Trapping When you read the output, two numbers stand out as very strange: 1. The objective value is **680.110**. As explained, in pFBA, the solver changes its goal from “Maximize Growth” to “Minimize total effort.” The mutant cell’s total sum of fluxes required to survive is 680.110, while its actual growth rate remains 0.704. 2. The flux through the alternate pathway is **27.90**. *How can a cell eating only 10 units of sugar process 27.90 units through a single enzyme?*

This reveals a fascinating phenomenon called **Stoichiometric Trapping**: Because the PFK enzyme is dead, a sugar intermediate called *Fructose-6-Phosphate (F6P)* cannot move forward through its normal metabolic door. It is trapped. To survive, the cell diverts the incoming 10 units of glucose into an alternate detour (the Pentose Phosphate Pathway, starting with **G6PDH2r**).

However, part of this alternate pathway spits out F6P. Because the forward door is still bricked shut, the F6P will build up and kill the cell. To avoid this, the cell frantically runs another

enzyme in reverse, turning the trapped F6P *back* into starting sugar, and shoving it right back into **G6PDH2r** again!

To process the 10 units of food coming from the environment, this pathway has to operate like a metabolic whirlpool, recycling the trapped sugars and spinning nearly three times as fast (roughly $3 \times 10 = 30$) just to keep the lights on.

0.0.5 Project Summary

Using simple matrix algebra and constraints (FBA), our tiny project just successfully predicted: 1. The baseline growth rate of *E. coli*. 2. The exact metabolic shift from respiration to fermentation (producing alcohol) when oxygen is removed. 3. The alternate biological pathways the cell will use to survive a genetic mutation.

This exact methodology is used today by pharmaceutical companies to engineer yeast that produce insulin, and by green-tech companies to engineer bacteria that eat plastic or produce biofuels!